

Rewiring Intelligence: The Evolution of Early Neural Networks from McCulloch-Pitts to PDP

Ishan Mysore, ATCS: Neural Networks, Period 3

September 19, 2025

1 An Introduction to Brain-like Computing

The conceptual foundations of neural networks were laid in 1943 at the University of Chicago, when neurophysiologist Warren McCulloch and mathematician Walter Pitts published "A Logical Calculus of the Ideas Immanent in Nervous Activity" [1]. In this landmark work, they introduced the first artificial neuron model, the **McCulloch-Pitts (MCP) neuron**, as a way to explain how networks of simple brain-like units could generate complex behavior [1]. Their model marked a departure from conventional sequential computing.

The MCP neuron itself was strikingly simple: it received binary inputs (0 or 1), summed them with fixed excitatory or inhibitory weights, and fired an output of 1 if the sum exceeded a threshold or 0 otherwise [1]. With its Boolean structure, it effectively acted as an AND/OR gate.

This simplicity proved to be effective. McCulloch and Pitts showed that networks of such threshold units could, in principle, compute any logical function and even implement memory through feedback loops [2]. Their work ignited interest in **brain-like computing**, as the brain's massively parallel architecture offered a radically different model for computing at a time when sequential, vacuum tube-based machines faced serious physical constraints [2].

This foundation was advanced in 1949 by Canadian neuropsychologist Donald Hebb's *The Organization of Behavior*. Hebb proposed that "neurons that fire together, wire together" (i.e., when two neurons repeatedly activate together, the connection between them grows stronger). This principle, also known as **Hebbian Learning**, suggested how networks might learn and adapt through experience, building upon the fixed logical architectures seen in the McCulloch-Pitts model.

2 Rosenblatt's Revolutionary Perceptron

Inspired by Hebb, psychologist Frank Rosenblatt sought to provide a specific mathematical implementation of Hebbian learning. Working at the Cornell Aeronautical Laboratory, he introduced the **perceptron** in 1957-58 as the first practical neural network capable of learning from data [3]. He directly addressed the main limitation of MCP neurons (their inability to adapt) by designing a system that could adjust its connection weights based on experience [4].

The perceptron operates as a **feedforward network**: a system where information flows strictly in one direction from inputs through any hidden layers to the output, without any feedback loops

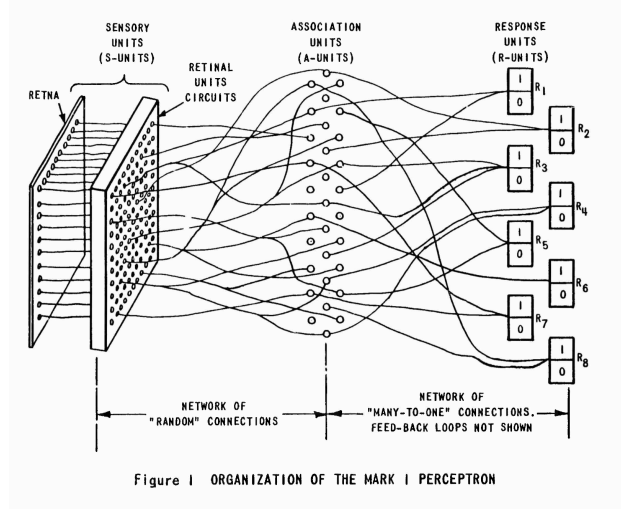


Figure 1: Mark I Perceptron [4]

[4]. Mathematically, each input x_i is multiplied by its corresponding weight w_i ; these weighted inputs are summed with a bias term b , and the result passes through an activation function f to produce output y :

$$y = f \left(\sum_i w_i x_i + b \right) \quad (1)$$

In Rosenblatt's original formulation, the perceptron used a **binary step** as its activation. When the weighted sum of inputs plus bias exceeded zero, the output was 1; otherwise, it was 0:

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases} \quad \text{where } z = \sum_i w_i x_i + b. \quad (2)$$

The perceptron's breakthrough was its ability to modify weights and bias whenever it misclassified an input, gradually improving performance over successive training examples [5]. This adaptive process transformed neural networks from fixed logical devices into machines that could *learn*.

To apply his mathematical idea in hardware, Rosenblatt built the **Mark I Perceptron** around 1958. As shown in Figure 1, this room-sized machine consisted of three functional layers: a sensory input layer of 400 photoelectric cells, an association (hidden) layer with randomly connected units, and a response (output) layer [6].

In one famous demonstration, an IBM 704 computer running the perceptron learning rule successfully learned to distinguish whether a card was marked on the left or right side after only 50 training trials [7]. Rosenblatt hailed this achievement, declaring the Mark I perceptron "the first machine which is capable of having an original idea." He envisioned that future perceptrons could recognize complex shapes, read text, or even play strategic games [7].

Rosenblatt's contribution culminated in the perceptron convergence theorem (1958), which proved that if training data are **linearly separable** (different classes can be separated by a straight line, or hyperplane in higher dimensions), the algorithm will find appropriate weights [7]. This theorem provided the first mathematical basis behind the way in which neural networks learn, in turn establishing neural networks as a legitimate field of study.

3 Minsky and Papert’s Critique and the AI Winter

Despite initial enthusiasm, Rosenblatt’s perceptron faced a mathematical critique. In 1969, Marvin Minsky and Seymour Papert published *Perceptrons: An Introduction to Computational Geometry*, in which they proved that single-layer perceptrons cannot compute certain basic logical functions, most famously the exclusive-OR (XOR) [9]. XOR exemplifies the challenge of non-linear separability: given inputs (0, 0), (0, 1), (1, 0), (1, 1) with corresponding outputs 0, 1, 1, 0, no single linear boundary can separate the “true” outputs (0, 1), (1, 0) from the “false” ones (0, 0), (1, 1).

Their analysis focused on what they termed **simple perceptrons**: networks with one input layer connected directly to one output neuron [8][9]. They demonstrated that such architectures can only represent convex decision regions, rendering them incapable of solving many important problems. For example, perceptrons cannot recognize whether an image depicts a connected shape or multiple disconnected parts [8][9]. These results revealed that, without hidden layers, perceptrons were mathematically incapable of capturing the complexity required for many real-world tasks.

The publication of *Perceptrons* had an immediate devastating effect on neural network research. As one retrospective notes, “skeptics insisted the perceptron was incapable of reshaping the relationship between human and machine. Enthusiasm waned” [10]. Government funding agencies, especially in the United States, redirected support away from machine learning approaches [11].

As a result, the first **AI Winter** followed, lasting about a decade [10][11]. The irony was that Minsky and Papert explicitly acknowledged that multilayer perceptrons were theoretically more powerful, but argued that no efficient training algorithms existed at the time [11]. Without a practical means of training deeper networks, their critique discouraged further investment precisely when the field required deeper exploration in order to grow. Rosenblatt himself, facing waning support, shifted his focus to other brain-inspired projects before his untimely death in 1971 [11]. As a result, neural networks remained a marginal pursuit until new mathematical tools and computational resources reignited interest in later decades.

4 The Revival: Backpropagation and Gradient Descent

A potential solution to Minsky and Papert’s critique emerged in 1974. Paul Werbos, then a Harvard graduate student, described the backpropagation algorithm in his dissertation as a method to train multilayer neural networks [12].

Backpropagation, as shown in Figure 2, employs the chain rule of calculus to compute how each weight in a multilayer network contributes to the final error [13]. The algorithm works by propagating error signals backward through the network: starting from the output layer’s error, it calculates how much each hidden layer weight should be adjusted to reduce that error [13]. Mathematically, this involves computing partial derivatives of the error function with respect to each weight, enabling **gradient descent**: a systematic method for finding the optimal weights by following the steepest path downhill in the error landscape [13].

Unfortunately, in light of the AI winter, Werbos’s work remained largely overlooked for more than a decade. However, in the mid-1980s, David Rumelhart, Geoffrey Hinton, and Ronald Williams independently rediscovered and popularized backpropagation at UCSD [14]. Their work demonstrated that multilayer networks with nonlinear activation functions could overcome the perceptron’s limitations. More specifically, backpropagation-trained networks could solve the XOR

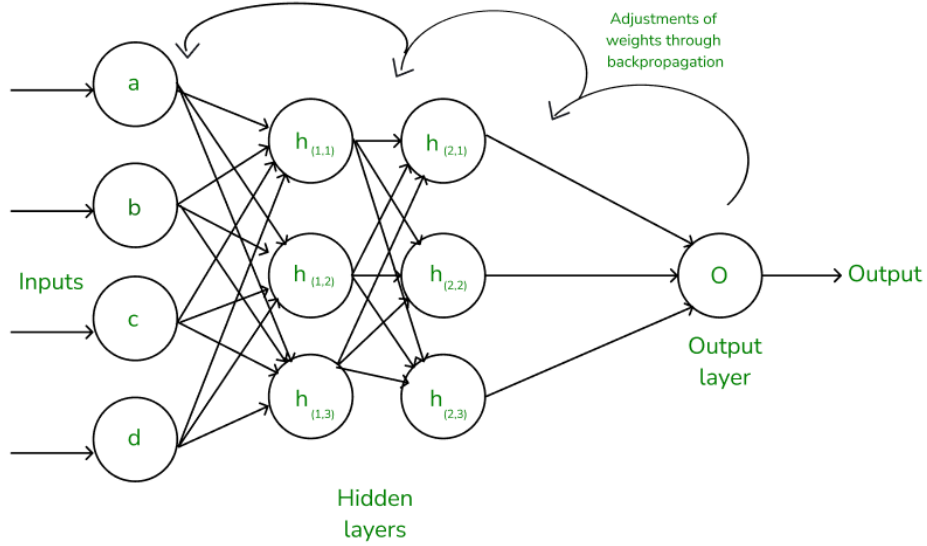


Figure 2: Backpropagation Algorithm [13]

problem, the very task that had doomed the original perceptron [13][14].

5 The Arrival of Parallel Distributed Processing

By 1986-87, David Rumelhart and James McClelland consolidated their insights in the landmark two-volume work *Parallel Distributed Processing: Explorations in the Microstructure of Cognition* [15]. This publication formalized the **PDP framework**, offering a comprehensive vision of how complex cognition might arise from the collective interactions of many simple processing units [15].

The PDP framework introduced several innovations that advanced earlier models. Chief among them was the replacement of rigid binary step functions with smooth, continuous sigmoid activations [16]. The **sigmoid** function,

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (3)$$

produces outputs between 0 and 1 that vary gradually with input, thereby making neural units differentiable. This change allowed networks to fine-tune adjustments through backpropagation and learn more complex mappings than step-based systems could achieve.

This "second wave" of connectionism (a term often used interchangeably with PDP) produced remarkable results. Neural networks began tackling speech recognition and natural language processing with unprecedented levels of performance [17]. These foundations were further strengthened by the **universal approximation theorem**, which established that multilayer perceptrons with sufficiently many hidden units could approximate any continuous function to arbitrary precision [16][18]. This result illustrated that neural networks were not only inspired by the brain but also mathematically capable of solving a wide range of problems. Combined with practical algorithms such as backpropagation, these developments restored confidence in connectionism and

drew a new generation of researchers into the field [16][17][18].

6 Improvements in PDP Architecture

6.1 Activation Function Evolution

Rosenblatt’s binary step function was replaced by smooth, nonlinear activations that made gradient descent possible. The sigmoid function dominated early PDP work, but sigmoids also introduced mathematical challenges: their gradients vanish for very large or very small inputs, slowing training in deep networks. Researchers addressed this with alternative activations. The hyperbolic tangent (tanh) function, which outputs values in the range $(-1,1)$, often accelerated convergence by centering activations around zero [19]. Later, the rectified linear unit (ReLU), defined as

$$f(x) = \max(0, x), \tag{4}$$

was extremely influential: it trains much faster than sigmoids, mitigates the vanishing gradient problem, and promotes sparse representations by deactivating some neurons entirely [20]. Variants such as Leaky ReLU, ELU (Exponential Linear Unit), and GELU (Gaussian Error Linear Unit) further improve stability and performance in deep models [20].

6.2 Learning Algorithm Advances

Beyond basic backpropagation, researchers developed more sophisticated training methods. Second-order optimization methods, such as Newton’s method and quasi-Newton approaches, leveraged curvature information from the Hessian matrix to adjust step sizes adaptively, leading to faster and more reliable convergence compared to simple gradient descent [19][21].

Equally important were regularization techniques designed to combat **overfitting**, which occurs when the performance of the model on training data beats its performance on testing data [22]. For instance, weight decay penalized large parameter values, effectively shrinking unneeded connections. In addition, early stoppage terminated training to prevent models from memorizing nonexistent patterns, in turn improving generalization [22][23]. Later methods such as dropout randomly silenced neurons during training, forcing networks to develop robust and redundant representations of the data [24].

6.3 Expanding Beyond Supervised Learning

The PDP framework naturally extended to **unsupervised learning**, where networks discover patterns without explicit training labels [25]. Reinforcement learning variants allow networks to learn from rewards and punishments rather than correct/incorrect examples [26]. These extensions broadened neural networks’ applicability far beyond pattern recognition into domains like game playing, robotics, and autonomous systems [25][26].

7 Modern Legacy: From PDP to Deep Learning

ChatGPT and image recognition systems today, despite their sophistication, still inherently rely on weighted sums and nonlinear activations—the same ideas Rosenblatt introduced sixty years ago. Convolutional Neural Networks (CNNs) utilize specialized architectures for processing images but make use of the same backpropagation learning Werbos and Rumelhart created. Recurrent Neural Networks (RNNs) add feedback connections to facilitate sequential processing while retaining the distributed processing approach of PDP. Even large language model transformer architectures utilize attention mechanisms that compute weighted sums of inputs, a direct descendant of the weighted sum operations seen in Rosenblatt’s perceptron.

The increased computational power in the 21st century has enabled researchers to train networks with millions or billions of parameters, yet the core insight remains unchanged: intelligence can emerge from the interaction of many simple, adaptive processing units. The brain-inspired vision that motivated McCulloch, Pitts, Hebb, and Rosenblatt continues to guide AI research today.

References

- [1] McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biology*, vol. 52, no. 1–2, pp. 99–115, 1943, doi: [https://doi.org/10.1016/s0092-8240\(05\)80006-0](https://doi.org/10.1016/s0092-8240(05)80006-0).
- [2] S. B. Furber, “Brain-inspired computing,” *IET Computers & Digital Techniques*, vol. 10, no. 6, pp. 299–305, Nov. 2016, doi: <https://doi.org/10.1049/iet-cdt.2015.0171>.
- [3] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: <https://doi.org/10.1037/h0042519>.
- [4] “Rosenblatt’s Perceptron.” Available: D. Haykin, *Neural Networks and Learning Machines*, 3rd ed., sample chapter: “Rosenblatt’s Perceptron”, Pearson Higher Education, 2009. [Online]. Available: <https://www.pearsonhighered.com/assets/samplechapter/0/1/3/1/0131471392.pdf> [Accessed: Sep. 17, 2025].
- [5] N.J. Nilsson, *Learning machines*, McGrawHill: New York, 1965.
- [6] Rosenblatt, F. (1962). *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Washington, D.C.: Spartan Books.
- [7] Olazaran, M. (1996). “A Sociological Study of the Official History of the Perceptrons Controversy.” *Social Studies of Science*, 26(3), 611–659.
- [8] H. D. Block, “A review of “perceptrons: An introduction to computational geometry,” *Information and Control*, vol. 17, no. 5, pp. 501–522, Dec. 1970, doi: [https://doi.org/10.1016/s0019-9958\(70\)90409-2](https://doi.org/10.1016/s0019-9958(70)90409-2).

- [9] “History: The Past,” University of Louisiana at Lafayette, 2025. [Online]. Available: <https://userweb.ucs.louisiana.edu/isb9112/dept/phil341/histconn.html>. [Accessed: Sep. 17, 2025].
- [10] Lefkowitz, M. (2019, September 25). Professor’s perceptron paved the way for AI – 60 years too soon. Cornell Chronicle. Retrieved from <https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon>
- [11] M. Richards, “From boom to bust: the AI winter,” Open University OpenLearn, Jan. 29, 2024. [Online]. Available: <https://www.open.edu/openlearn/digital-computing/from-boom-bust-the-ai-winter>. [Accessed: Sep. 17, 2025].
- [12] P. J. Werbos, Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences. PhD thesis, Harvard University, 1974.
- [13] “Backpropagation Algorithm - an overview — ScienceDirect Topics,” <https://www.sciencedirect.com/topics/computer-science/backpropagation-algorithm>
- [14] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning Representations by Back-Propagating Errors,” The MIT Press eBooks, pp. 213–222, Sep. 2002, doi: <https://doi.org/10.7551/mitpress/1888.003.0013>.
- [15] Rumelhart, D. E., & McClelland, J. L. (Eds.). (1986). Parallel distributed processing: Explorations in the microstructure of cognition (Vols. 1-2). MIT Press.
- [16] Cybenko, G. Approximation by superpositions of a sigmoidal function. Math. Control Signal Systems 2, 303–314 (1989). <https://doi.org/10.1007/BF02551274>
- [17] J. J. Hopfield, “Neural networks and physical systems with emergent collective computational abilities,” Proceedings of the National Academy of Sciences, vol. 79, no. 8, 1982.
- [18] K. Hornik, M. Stinchcombe & H. White, “Multilayer Feedforward Networks are Universal Approximators,” Neural Networks, 1989; also “Approximation Capabilities of Multilayer Feedforward Networks” (1991).
- [19] LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (1998). Efficient BackProp. In Neural Networks: Tricks of the Trade (pp. 9–50). Springer.
- [20] R. Parhi and R. D. Nowak, “The Role of Neural Network Activation Functions,” IEEE Signal Processing Letters, vol. 27, pp. 1779–1783, 2020, doi: <https://doi.org/10.1109/lsp.2020.3027517>.
- [21] Pearlmutter, B. A. (1994). Fast exact multiplication by the Hessian. Neural Computation, 6(1), 147–160.
- [22] Krogh, A., & Hertz, J. A. (1992). A simple weight decay can improve generalization. Advances in Neural Information Processing Systems, 4, 950–957.

- [23] Prechelt, L. (1998). Early stopping — but when?. In G. Orr & K. Müller (Eds.), *Neural Networks: Tricks of the Trade* (pp. 55–69). Springer.
- [24] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- [25] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- [26] Sutton, R. S., & Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. MIT Press.